

MIPS Instruction Representation and Addressing

1 Instructions as Data

- Instructions are binary data stored in memory.
- The processor fetches instructions from memory to execute them.
- The **Program Counter (PC)** is a special register that points to the next instruction to fetch.
- Instructions must be encoded in binary: this is called **machine code**.

2 MIPS Instruction Encoding Overview

- Every MIPS instruction is exactly 32 bits long.
- Three main instruction formats:
 - **R-format (Register)**
 - **I-format (Immediate)**
 - **J-format (Jump)**
- Other architectures may use variable-length instructions (more flexible, but hardware is complex).

3 R-format Instructions

Fields:

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

- **op**: Operation code.

- **rs, rt**: Source registers.
- **rd**: Destination register.
- **shamt**: Shift amount (used for shift instructions).
- **funct**: Extends the opcode for specific operations.

Example:

```
add $t0, $s1, $s2
Binary: 000000 10001 10010 01000 00000 100000
Hex:    0x02324020
```

4 I-format Instructions

Used for:

- Immediate arithmetic.
- Load/store instructions.

op	rs	rt	Immediate/Offset
6 bits	5 bits	5 bits	16 bits

Example 1: addi

```
addi $s1, $s2, 100
Binary: 001000 10010 10001 00000000001100100
```

Example 2: lw

```
lw $s1, 100($s2)
Binary: 100011 10010 10001 00000000001100100
```

Example 3: sw

```
sw $s1, 100($s2)
Binary: 101011 10010 10001 00000000001100100
```

5 Branch Addressing

- Branches use PC-relative addressing.
- Offset is a signed 16-bit value, multiplied by 4.
- Target address:

$$\text{Target} = (PC + 4) + (\text{offset} \times 4)$$

- Useful for nearby conditional jumps and it's use I-Format Instructions.

Branch Example

```
bne $t0, $s5, Exit
Addresses:  PC = 0x1408000C,  Exit = 0x14080018
Offset =  $\frac{0x14080018 - (0x1408000C + 4)}{4} = 2$ 
```

6 J-format Instructions

op	address
6 bits	26 bits

- Used for j, jal.
- Address is calculated as:

$$\text{Target} = PC_{31-28} : (\text{address} \times 4)$$

- Allows jump within the same 256 MB region.

Jump Example

```
j Loop
```

```
Target:  0x14080000 = 0x1 : (0x1020000 × 4)
```

7 Branching Far Away

- If target is too far for 16-bit offset:

```
beq $s0, $s1, L1
... # code
L1:
```

- Convert to:

```
bne $s0, $s1, L2
j L1
L2:
```

8 Jumping Far Away

- If jump target is too far for 26-bit address:

```
j L1          # too far
...
L1:
```

- Convert to:

```
la $t0, L1
jr $t0
...
L1:
```